Multimodal AI

Multimodal AI refers to artificial intelligence systems or models that can process and integrate information from multiple modalities or sources of data. These modalities can include text, images, videos, audio, and other forms of sensory data.

Multimodal AI examples

Multimodal AI is advancing across different fields, combining multiple different types of data to create powerful and versatile outputs. A few notable examples include:

- **GPT-4V(ision)** is upgraded GPT-4 version that can process images as well as text, meaning the AI can generate visual content.
- **Inworld AI** can create intelligent and interactive virtual characters in games and other digital worlds.
- **Runway Gen-2** can use text prompts to generate dynamic video.
- **DALL-E 3** is an OpenAI-based model that generates high-quality images based on text prompts.
- **ImageBind by Meta AI** uses six data modalities — text, image, video, thermal, depth, and audio — to generate outputs.
- **Google's Multimodal Transformer (MTN)** combines audio, text, and images to generate captions and descriptive video summaries.

Multimodal AI tools

Several advanced tools are already paving the way for enhancing multimodal artificial intelligence.

•**Google Gemini** can integrate images, texts, and other modalities to create, understand, and enhance content.

•**Vertex AI** is the machine learning platform of Google Cloud. it can also process different data and perform tasks like image recognition, analyzing video, and more.

•**OpenAI's CLIP** can process text and images to perform tasks like visual search and image captioning.

•**Hugging Face's Transformers** can support multimodal learning and build versatile AI systems by processing audio, text, and images.

All these systems prove that multimodal AI is growing in the field of content creation, gaming, and dealing with other real-world scenarios.

How its work?

Multimodal AI works by integrating and processing information from multiple modalities (such as text, image, audio, and sometimes other sensor data) to perform tasks or make decisions. Here's an overview of how multimodal AI typically operates:

Data Integration

Multimodal AI systems take inputs from different modalities. These inputs can be in various forms:

•**Text**: Natural language inputs, such as typed queries or spoken commands.

•**Image**: Visual inputs, including photos, videos, or live camera feeds.

•**Audio**: Spoken language or sound inputs, processed as speech.

•**Other Sensors**: Data from additional sensors like temperature, motion, etc., depending on the application.

Feature Extraction

Each modality requires specific feature extraction techniques to convert raw data into a format that the AI model can understand and process. For example:

•**Text**: Natural language processing (NLP) techniques tokenize, parse, and extract semantic meaning from text.

•**Image**: Computer vision techniques extract features like shapes, colors, objects, and spatial relationships.

•**Audio**: Signal processing and speech recognition techniques convert audio signals into text or phonetic representations.

Integration and Fusion

Once features are extracted from each modality, they are integrated or fused together to create a unified representation of the input data. Fusion techniques can vary:

•**Early Fusion**: Combining raw data from different modalities before feeding it into the AI model.

•**Late Fusion**: Processing each modality separately and combining outputs at a later stage based on learned representations.

Modeling and Decision Making

The integrated representation is then used as input for AI models that perform specific tasks, such as:

•**Classification**: Determining the category or class of the input data (e.g., classifying objects in images or sentiments in text).

•**Generation**: Creating new content based on learned patterns from multiple modalities (e.g., generating captions for images).

•**Prediction**: Forecasting outcomes based on integrated data inputs (e.g., predicting trends using combined textual and visual data).

Feedback and Adaptation

Multimodal AI systems often incorporate feedback mechanisms to improve performance over time. This may involve retraining models with new data, adjusting fusion strategies, or refining feature extraction techniques based on user interactions and outcomes.

Deployment and Application

Applications of multimodal AI range from smart assistants that understand and respond to voice commands while displaying information visually, to healthcare systems that combine patient data from various sources for diagnosis and treatment planning.

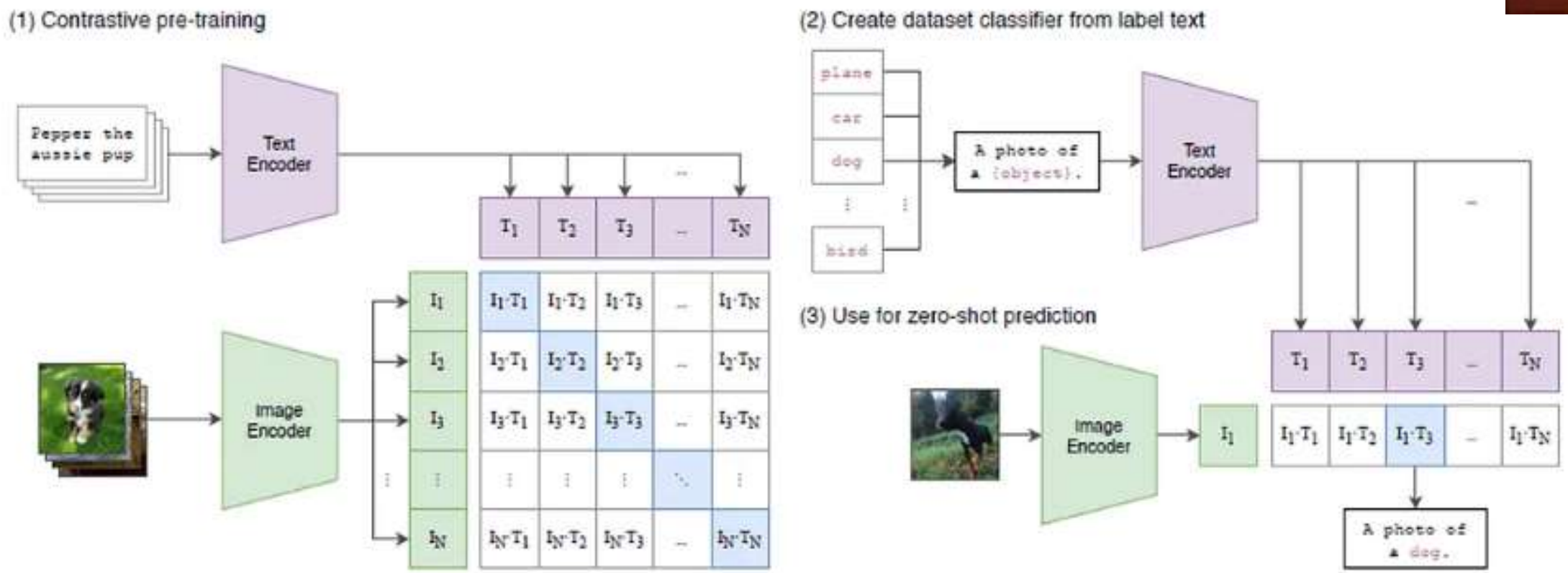
Exploring CLIP, BLIP

CLIP (Contrastive Language-Image Pre-training)
OpenAI’s CLIP was a game-changer.

It emphasizes the use of natural language as a powerful source of supervision for learning visual concepts.

It argues that leveraging the vast amounts of text available on the web can enhance the learning capabilities of visual models beyond the constraints of traditional image datasets.

1- Model architecture and pre-training



CLIP approach(source CLIP paper)

The CLIP architecture includes a text encoder and an image encoder, which are used to encode text and images, respectively.

Image encoder

•The paper discusses choosing between two types of architectures for the image encoder: a ResNet-50 base model and variations of Vision Transformers (ViT).

•The training involved five ResNet models and three Vision Transformer models. For ResNets: ResNet-50, ResNet-101, and three models scaled in the style of EfficientNet (RN50x4, RN50x16, RN50x64), which use 4x, 16x, and 64x the compute of a ResNet-50, respectively.

• For Vision Transformers: ViT-B/32, ViT-B/16, and ViT-L/14.

•The ViT-L/14@336px model, trained at a higher resolution, was identified as the best-performing model in the series and was used for reporting results in the study under the name “CLIP.”

Text encoder

•The text encoder is based on a Transformer architecture, incorporating modifications from Radford et al. (2019). It consists of a 12-layer structure with 512 units width and 8 attention heads, totaling approximately 63 million parameters.

•**Text embedding**: It uses byte pair encoding (BPE) for text embedding, with a vocabulary size of 49,152, which helps in handling a variety of textual inputs efficiently.

•**Sequence Handling**: The maximum sequence length for processing is capped at 76 tokens to optimize computational efficiency. Text sequences are processed by enclosing them with special tokens for start of sequence ([SOS]) and end of sequence ([EOS]).

Training

•The CLIP training objective is contrastive learning, where an image encoder and a text encoder are trained to maximize the similarity between embeddings of correct image-text pairs (blue cells in similarity matrix in the below figure) while minimizing it for incorrect pairs (white cells in the matrix in the figure). This is accomplished by optimizing a symmetric cross-entropy loss based on these similarities.

•**Image-Text Pairs**: The dataset is comprised of pairs where each image is directly associated with a relevant text description. These could be captions, titles, or other forms of descriptive text that accurately represent the visual content of the image.

•**Source of Data**: These pairs are typically sourced from the internet, databases, or datasets specifically designed for tasks involving multimodal learning (image and text). Examples include datasets compiled from websites where images are posted with captions, or curated datasets where annotations are provided by human labelers.

•**Projection Method**: The model employs a linear projection to map the representations from each encoder to the multi-modal embedding space, avoiding more complex non-linear projections that did not show a noticeable benefit in terms of training efficiency.

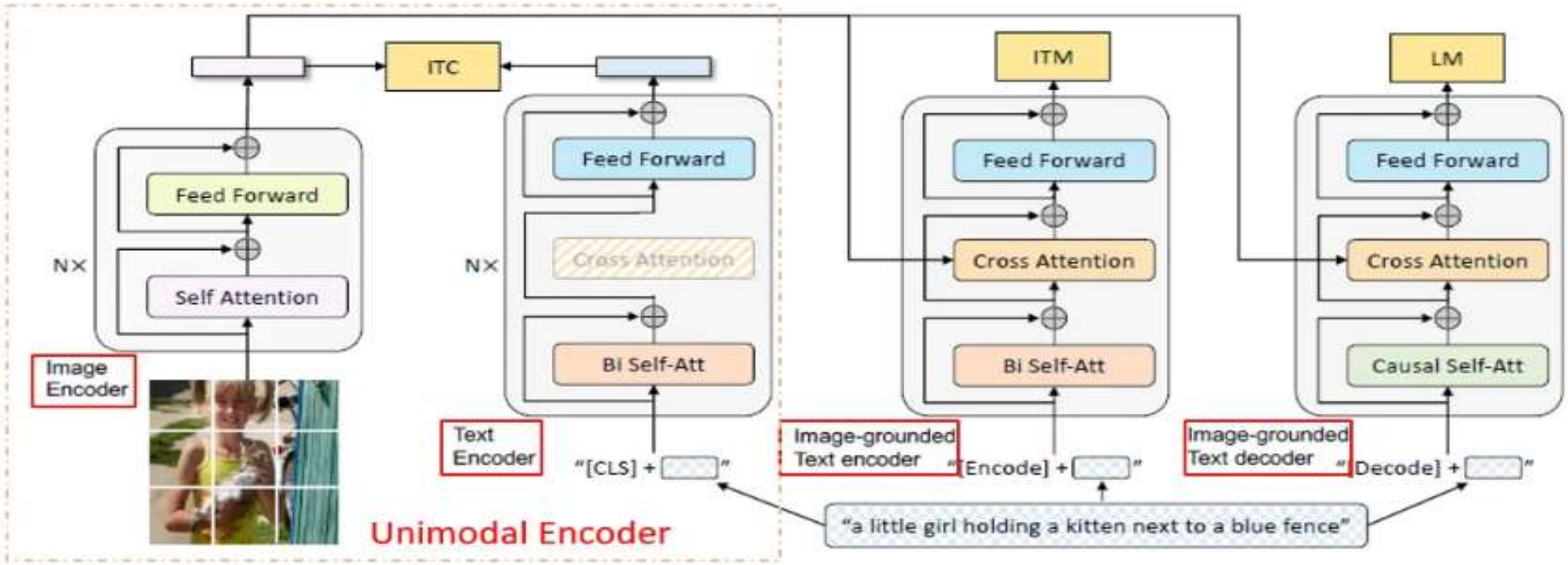
BLIP (Bootstrapping Language-Image Pre-training)

High-quality human-annotated image-text pairs are limited due to the prohibitive cost of annotation (e.g., COCO dataset) therefore recent methods use a larger number of image and alt-text pairs{f(Iw; Tw)} collected from the web.

Alt-texts from web data often do not accurately describe the visual content of the images, making them noisy signals that are suboptimal for learning effective vision-language alignment.

To address this issue, BLIP introduces a new method called CapFilt, which filters out noisy pairs and generates new captions for images. The following sections discuss the model architecture, as well as the pre-training and fine-tuning processes.

1- Model architecture and pretraining in BLIP



Pre-training model architecture of BLIP (source BLIP paper)

Unimodal Encoder:

•Separately encodes images and text. The text encoder functions similarly to BERT, appending a [CLS] token to the beginning of the text input to summarize the sentence.

•Visual transformer (ViT) is used as image encoder which divides an image into patches, encoding them as a sequence of embeddings, including an additional [CLS] token to represent the global image feature.

•It is **trained** with an image-text contrastive (ITC) loss to align feature spaces of the image encoder and text encoder. Similar to CLIP, ITC encourages positive image-text pairs to have similar representations and distinguishes them from negative pairs.

Image-grounded Text Encoder:

•Adds a cross-attention layer between the self-attention layer and the feed-forward network in the text encoder, to integrate visual information.

•A task-specific [Encode] token is appended to the text, and the output embedding of [Encode] is used as the multimodal representation of the image-text pair.

•It is **trained** with an image-text matching (ITM) loss. ITM is a binary classification task, where the model uses an ITM head (a linear layer) to predict whether an image-text pair is positive(matched) or negative (unmatched) given their multimodal feature.

Image-grounded Text Decoder:

•Replaces the bidirectional self-attention layers with causal self-attention layers. it uses a [Decode] token to signal the start of a sequence and an end-of-sequence token to mark its end.

•The decoder is **trained** with a language modeling (LM) loss to generate captions given images. Optimizes a cross-entropy loss to maximize the likelihood of text in an autoregressive manner.

Parameter Sharing: the architecture shares all parameters except for the self-attention layers between the text encoder and decoder to enhance training efficiency.

The differences between the encoding and decoding tasks are the SA layers. the encoder employs bi-directional self-attention to build representations for the current input tokens, while the decoder uses causal self-attention to predict next tokens.

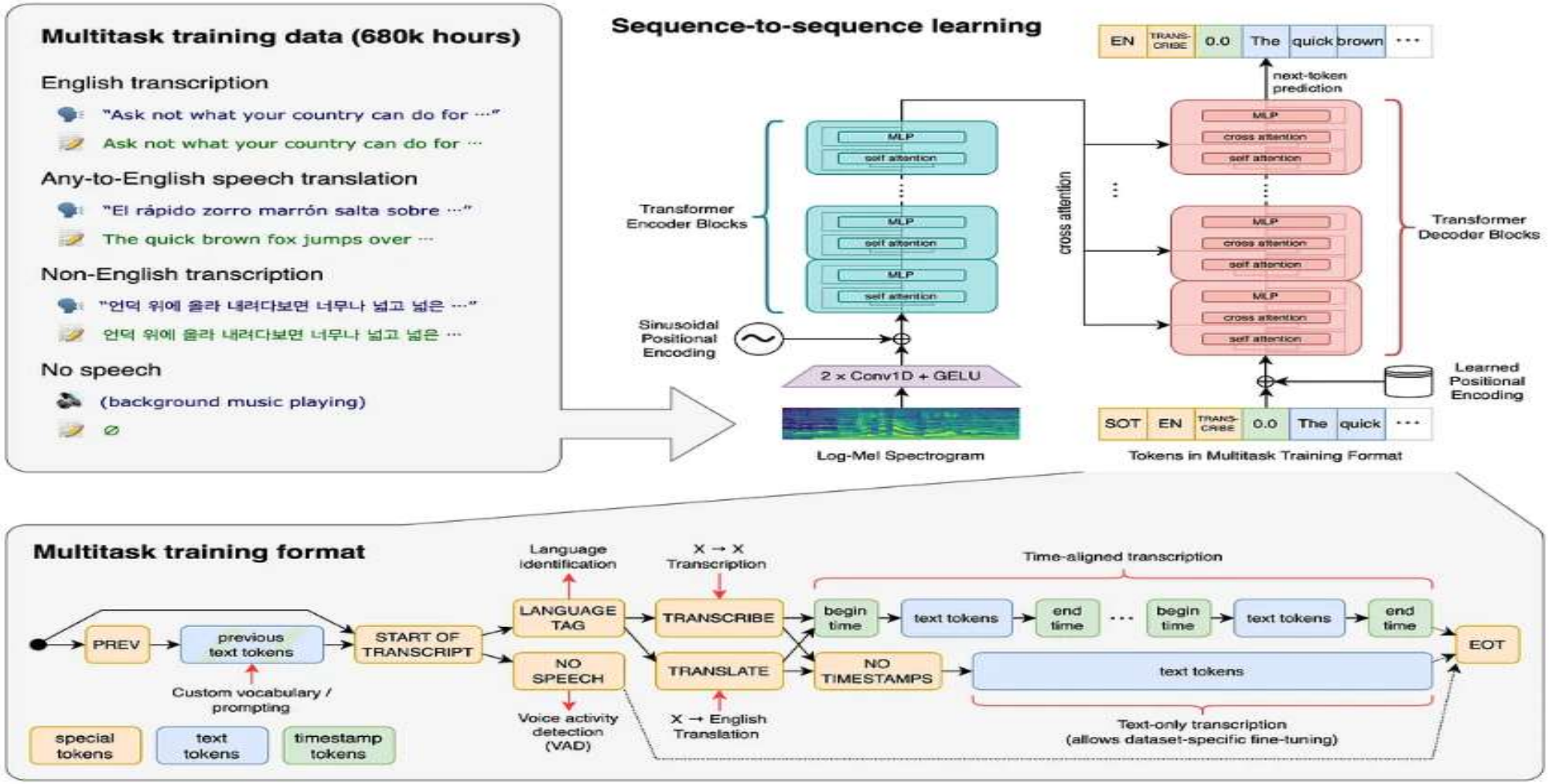
Introducing Whisper: Speech Recognition Model Capable of Recognizing 99 Languages

- Whisper is an automatic speech recognition (ASR) system trained on 680,000 hours of multilingual and multitask supervised data collected from the web.

- We show that the use of such a large and diverse dataset leads to improved robustness to accents, background noise and technical language.

- Moreover, it enables transcription in multiple languages, as well as translation from those languages into English. We are open-sourcing models and inference code to serve as a foundation for building useful applications and for further research on robust speech processing.

Architecture



Source: https://github.com/openai/whisper

The Whisper architecture is a simple end-to-end approach, implemented as an encoder-decoder Transformer. Input audio is split into 30-second chunks, converted into a log-Mel spectrogram, and then passed into an encoder.

A decoder is trained to predict the corresponding text caption, intermixed with special tokens that direct the single model to perform tasks such as language identification, phrase-level timestamps, multilingual speech transcription, and to-English speech translation.

How Does OpenAI Whisper Work?

Whisper is a complex system incorporating multiple deep learning models trained on a massive dataset of audio and text. Here's a simplified explanation on how it works:

1.Audio Preprocessing: The audio input is divided into short segments and converted into spectrograms (visual representations of audio frequencies).

2.Feature Extraction: Deep learning models extract relevant features from the spectrograms, capturing linguistic and acoustic information.

3.Language Identification: If the language is unknown, a separate model identifies it from supported languages.

4.Speech Recognition: A model trained on spoken language predicts the most likely sequence of words that corresponds to the extracted features.

5.Translation (Optional): If translation is requested, another model translates the recognized text into the desired language.

6.Post-processing: The output is refined using language rules and heuristics to improve accuracy and readability.

Benefits of Using OpenAI Whisper

•**High Accuracy:** Whisper achieves state-of-the-art results in speech-to-text and translation tasks, particularly in domains like podcasts, lectures, and interviews.

•**Multilingual Support:** It handles over 57 languages for transcription and can translate from 99 languages to English.

•**Robustness to Noise and Accents:** Whisper is relatively good at handling background noise, different accents, and technical jargon.

•**Open-Source Availability:** The model and inference code are open-source, allowing for customization and research contributions.

•**API and Cloud Options:** It has both a free command-line tool and a paid API for cloud-based processing, offering flexibility for different use cases.

•**Cost-Effectiveness:** The API pricing is competitive compared to other speech-to-text solutions.

What is an agentic workflow?

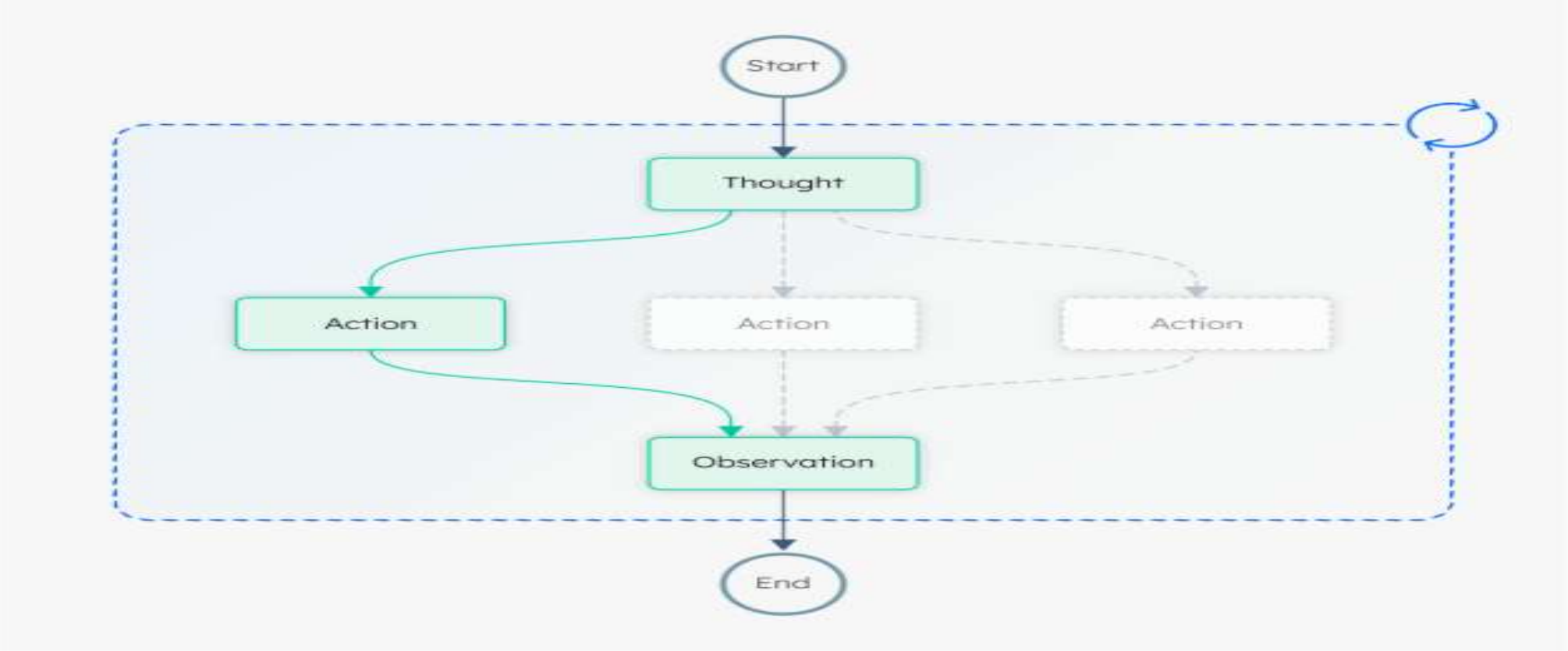
- An agentic workflow is an AI-driven process where the sequence of tasks are dynamically executed with minimal human intervention to achieve a particular goal.

- Agentic workflows are a form of agentic AI systems, which consist of AI models with memory, planning, reasoning, and tool-use capabilities.
- Unlike traditional automated workflows, agentic workflows are dynamic, enabling it to account for multiple unexpected variables and achieve more complex tasks that often exceed the boundaries of what an algorithmic script can do.

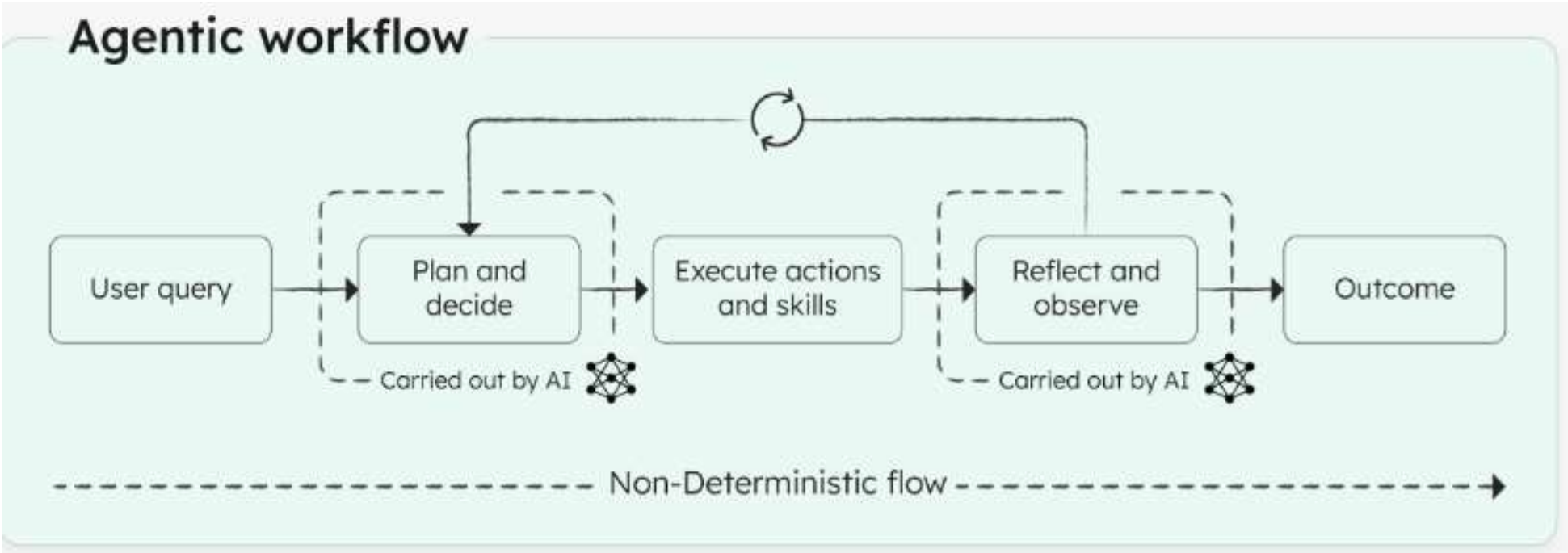
- Rather than following a predetermined path, an agentic workflow involves constant evaluation of what steps to take next, based on real-time information and conditions.

- To achieve this level of flexibility, agentic workflows usually involve a decision-making loop until the resolution is accomplished.

- At a high level, an agentic workflow can be distilled into a process that contains an iterative Thought–Action–Observation loop. In this loop, an AI model assesses the situation, devises or updates a plan, takes action (often via an external tool or API), observes the result, and so on.



Thought-Action-Observation loop in an agentic workflow.


Agentic workflow
User query
Plan and decide
Execute actions and skills
Reflect and observe
Outcome
Carried out by AI
Carried out by AI
Non-Deterministic flow

What is an agentic framework?

- An agentic framework provides the foundational structure for building autonomous AI systems that can reason, coordinate, and take action across enterprise environments.

-
- Rather than focusing on individual models or isolated tasks, an agentic framework defines how multiple AI agents interact with each other and with enterprise systems to achieve shared objectives.

- It establishes the rules, protocols, and architectural patterns that govern autonomous behavior at scale.

At its core, an agentic framework defines how agents:

- Communicate and exchange context
- Coordinate actions across workflows
- Reason about goals, constraints, and policies
- Make decisions and adapt based on outcomes

This structure allows organizations to move beyond isolated automations and build multi-agent systems that operate cohesively across enterprise platforms, including IT service management platforms (ITSM), HR systems, identity providers, and collaboration tools.

Key features of agentic frameworks

Pre-built components

Provide standardized building blocks—such as workflow templates, triggers, and system connectors—that accelerate the development and deployment of AI agents.

Communication protocols

Define how multiple AI agents exchange information and coordinate tasks. Help distribute work across specialized agents, enabling multi-agent workflows to be reliable, traceable, and aligned with business rules.

Planning and reasoning

Allow agents to move beyond rigid scripts by interpreting context, evaluating constraints (like policies and system states), and dynamically determining the appropriate sequence of actions to achieve a goal.

Monitoring and debugging

Provide essential visibility into how agents perform, track decisions, and surface bottlenecks or errors as workflows execute, for continuous observation and refinement.

Pre-built components

- Agentic frameworks typically include pre-built components that accelerate the development and deployment of AI agents while enforcing consistent agent behavior.
- These components provide standardized building blocks — such as workflow templates, triggers, actions, and system connectors — that reduce the need to design every agent from scratch.
- In enterprise settings, pre-built components enable teams to deploy agents more quickly and consistently across systems like IT service management platforms, HR tools, and identity providers.
- Instead of reimplementing reasoning and execution logic for every use case, organizations can reuse validated patterns to ensure agents behave predictably and align with operational and security requirements.
- By standardizing how agents reason, act, and integrate with systems, pre-built components help enterprises scale autonomy while maintaining reliability, consistency, and control.

Communication protocols

- Communication protocols define how AI agents exchange information, coordinate tasks, and hand work off to one another within an agentic system.

- In an enterprise context, these protocols are critical for ensuring that multi-agent workflows remain reliable, traceable, and aligned with business rules.

- Rather than relying on a single agent to perform every action, agentic frameworks use structured communication patterns to distribute work across specialized agents.

- Each agent focuses on a specific responsibility — such as validating inputs, executing actions, or enforcing policies — while the framework governs how information flows between them.

Planning and reasoning

- Planning and reasoning capabilities are what allow AI agents to move beyond scripted automation and operate autonomously within complex environments.
- Agentic frameworks incorporate reasoning engines that enable agents to interpret context, evaluate constraints, and determine the appropriate sequence of actions needed to achieve a goal.
- Instead of following rigid, pre-programmed workflows, agents can assess variables such as user role, system state, policies, and historical outcomes.
- This allows them to plan dynamically, adjust when conditions change, and handle exceptions without requiring constant human input.

Monitoring and debugging

- Monitoring and debugging capabilities are essential for maintaining trust and reliability in autonomous systems.
- Agentic frameworks typically include tools that allow teams to observe agent behavior, track decisions, and identify issues as workflows execute across enterprise environments.
- These capabilities provide visibility into how agents perform over time, surfacing bottlenecks, errors, or unexpected outcomes before they become systemic problems.
- For example, if an automated workflow consistently slows down due to missing data or approval delays, monitoring tools can help teams pinpoint where the breakdown occurs and adjust the system accordingly.
- Debugging tools also make it easier to evolve agentic systems safely.
- When changes are required such as updating policies, integrations, or decision logic, teams can test, trace, and refine agent behavior using real-time insights rather than trial-and-error fixes.

LLM Evaluation Techniques

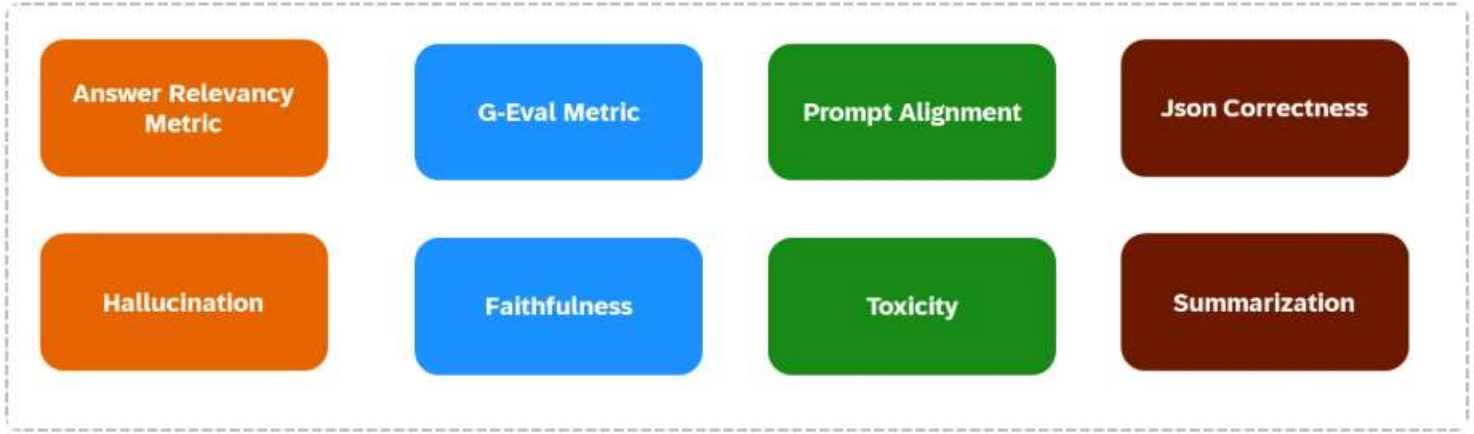
What is DeepEval?

- DeepEval serves as a comprehensive platform for evaluating LLM performance, offering a user-friendly interface and extensive functionality.

- It enables developers to create unit tests for model outputs, ensuring that LLMs meet specific performance criteria.

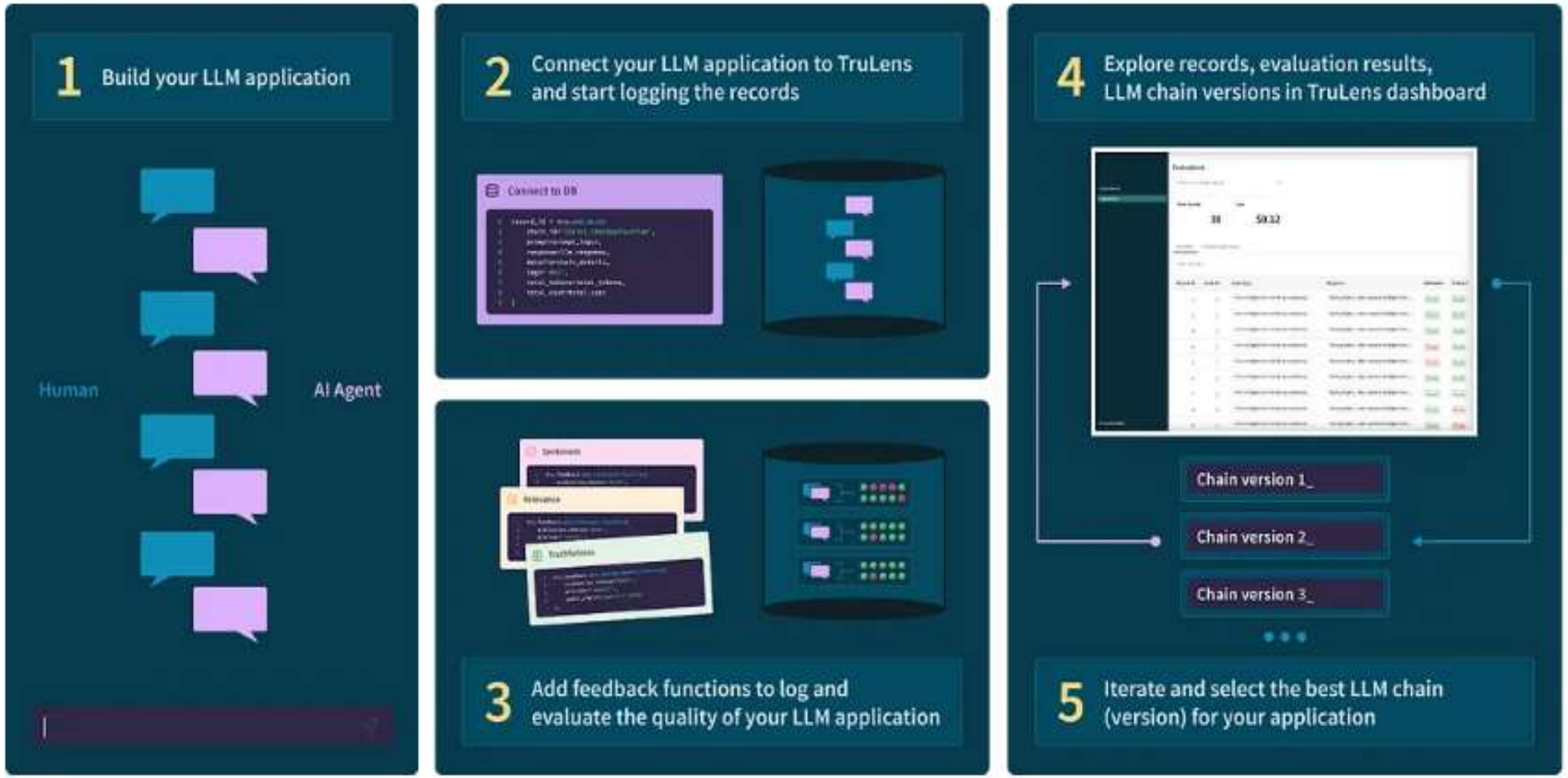
- The framework operates entirely on local infrastructure, which enhances security and flexibility while facilitating real-time production monitoring and advanced synthetic dataset generation.

Key Features of DeepEval



Some Metrics In DeepEval Framework

TruLens


1 Build your LLM application
Human
AI Agent
2 Connect your LLM application to TruLens and start logging the records
Connect to DB
3 Add feedback functions to log and evaluate the quality of your LLM application
4 Explore records, evaluation results, LLM chain versions in TruLens dashboard
Chain version 1_
Chain version 2_
Chain version 3_
5 Iterate and select the best LLM chain (version) for your application

LLM Security: Risks and Security Best Practices

Security Risks for LLM Applications

LLM01: Prompt Injection

LLM02: Insecure Output Handling

LLM03: Training Data Poisoning

LLM04: Model Denial of Service

LLM05: Supply Chain Vulnerabilities

Best Practices for Securing LLM Applications

1. Sanitize Inputs
2. 2. Data Minimization
3. 3. Data Encryption
4. 4. Implementing Access Control
5. 5. Auditing
6. 6. Secure Training Data
7. 7. API Security

Recent Developments in LLM